JOTFUZZ: Fuzzing the PHP JIT Optimizer by Satisfying Optimization Preconditions

***Abstract*—Just-in-time (JIT) compilation improves the performance of dynamic language runtimes, but its increased complexity introduces security risks. PHP is the most widely-used server-side programming language with vulnerabilities in its interpreter resulting in denial-of-service, sandbox escape, and remote code execution. Yet, existing fuzzers primarily exercise interpreter-level behaviors that unreliably activate JIT optimization, leaving JIT-specific logic insufficiently tested. In this paper, we present a template-guided generation approach for systematically fuzzing the PHP JIT optimizer. We present JIT Optimization Templates (JOTs), which encode the semantic preconditions required for optimization, including execution hotness, type stability, and control flow structure. Building on these templates, we design a JIT-aware fuzzing framework that matches seed fragments against optimizer constraints to reliably trigger optimized execution. Our evaluation on the PHP JIT demonstrates that JOTFUZZ uncovers 20 confirmed bugs in PHP 8.5.0-dev, including 14 in the JIT optimizer; 13 of the 14 JIT findings are security relevant, including memory-safety violations and denial-of-service faults. These results show that JIT optimizers can be systematically fuzzed by explicitly enforcing their semantic preconditions, using constraint-guided seed generation to reliably drive execution into optimized states that existing fuzzers rarely reach.**

I. INTRODUCTION

PHP's Just-In-Time (JIT) optimizer [1] is a complex, security-sensitive component that translates frequently executed code paths into optimized native code. Guided by runtime profiling, it performs speculative specialization and low-level transformations based on assumptions about types, control flow, and execution state. Because the optimizer is implemented in memory-unsafe C, violations of these assumptions can corrupt runtime state, leading to memory-safety vulnerabilities. Such vulnerabilities are particularly concerning for PHP's Internet-facing and multi-tenant deployments, where interpreter compromise may ultimately enable denial of service or remote code execution.

Fuzzing the PHP JIT poses challenges beyond conventional interpreter fuzzing, because reaching the code under test requires clearing two successive stages. *(Stage 1)* A generated program must first execute as PHP raises fatal errors on undefined variables, functions, and classes, and a program that aborts during compilation or early execution will fail to reach optimization. *(Stage 2)* Having executed, it must then activate the optimizer, requiring a combination of hotness, type stability, and control flow stability. Until both hold, execution remains interpreted and the optimizer is never exercised. Reaching the JIT is thus itself a prerequisite for testing it.

Existing fuzzers are poorly aligned with this requirement. Traditional coverage-guided approaches rely on mutation and typically generate or inherit shallow programs that do not reliably satisfy JIT preconditions. PHP's test suite provides limited structural support. Although it contains roughly 500 JIT-specific tests, over 96% of the broader PHP test corpus exhibits purely sequential control flow [2]. Consequently, fuzzing campaigns can achieve substantial interpreter coverage while exercising relatively little optimized execution, leaving the JIT optimizer under-tested.

Listing 1: A typical PHP test with sequential control flow. No code region becomes hot enough for JIT compilation.

```
<?php
$inputs = array(
    '<frameset></frameset>',
    '<html><frameset></frameset></html', );
$t = tidy_parse_string($inputs[0]);
$t->cleanRepair();
var_dump(tidy_get_body($t));
```

Motivating Example. Consider the PHP script in Listing 1, representative of programs generated by conventional mutation-based fuzzers and typical of PHP's own test suite. The script performs a series of array operations and function calls in a purely sequential manner, with no loops, repeated invocations, or structured control flow. When executed with PHP's JIT compiler enabled, the Zend engine interprets this script entirely without triggering optimization, since no code region satisfies the hotness thresholds required for JIT compilation. Though this script is valid, it fails at the second stage. Validity and activation are independent properties, and a fuzzer that achieves the first does not automatically obtain the second.

Activation as a Construction. Our central design decision is to treat second-stage reachability as a requirement of generation instead of a coincidence. We generate each program from an explicit description of the conditions that gate engagement including hotness, control flow, and repeated invocations. Every program the system generates satisfies such a description by construction. The consequence is that activation ceases to be a stochastic event and becomes a property of the fuzzer.

Syntax Agnostic Templates. These gating conditions are properties of a program's shape and execution profile, not of underlying syntax. JOTs capture these properties while discarding instantiation-specific syntax, making JOTs independent of their sourced languages. Consequently, optimizer test suites become interchangeable sources of JIT-triggering structure. A small optimizer test corpus can be augmented using JOTs extracted from another language. This transfer

does not assume that the source and target JITs optimize identically. Rather, the JOT preserves the relevant structural properties and re-parameterizes runtime-specific thresholds during instantiation.

JOTFUZZ. Based on this insight, we present JOTFUZZ. JOTFUZZ parses JavaScript engine tests and PHP JIT tests into ASTs and abstracts each one into a *JIT Optimization Template* (JOT): a syntax-agnostic tree that records control flow shape and data operation types. A JOT is then instantiated in PHP by filling each slot with real source fragments drawn from the PHP test suite. The usage of syntax-agnostic templates allows for the re-use of historically vulnerable PHP patterns and allows augmentation by vulnerable JavaScript structures. Concerete ly, V8 provides 5,698 JOTs compared with 475 from PHP's JIT tests, giving JOTFUZZ substantially more optimizer-triggering structures to instantiate in PHP.

PHP raises fatal errors on undefined variables, functions, and classes, and a seed that aborts early is useless regardless of how well its structure matches the constraint, because it dies before any code region can become hot. JOTFUZZ therefore precomputes, for every statement in its corpus, a *dependency slice* which is the transitive closure of statements needed to define everything that statement references. Filling a slot emits the whole slice, so structural fidelity and executability hold simultaneously.

Contributions. In short, this paper makes the following contributions:

- **JOT: JIT Precondition Modeling Templates.** A novel, structured representation of optimizer-triggering preconditions over control flow, data flow, and execution hotness that ensures JIT activation.
- **A Fuzzing Framework Based on JIT Constraints.** A mutation engine built to satisfy the JOT constraints that enables direct fuzzing of the JIT.
- **Extensive Empirical Evaluation on the Official PHP Interpreter.** A thorough evaluation demonstrating improved JIT activation and the discovery of numerous previously unknown vulnerabilities compared to state-of-the-art fuzzers.

II. BACKGROUND

The PHP Interpreter. The PHP interpreter is implemented in millions of lines of memory-unsafe C and can be decomposed into three subsystems: the Zend engine (*zend* directory), the core modules (*ext* directory), and the main functions (*main* directory) [2]. The Zend engine is the central execution engine, translating PHP scripts into intermediate opcodes that are executed by the interpreter. The core modules are responsible for functionality beyond the main interpreter like regular expression parsing. The main functions handle PHP runtime code and are responsible for initializing and executing the interpreter. Notably, the interpreter integrated a Just-In-Time compiler in 2019 [1], which detects and optimizes frequently executed code paths.

JIT Optimization. Just-In-Time (JIT) compilation is widely adopted in dynamic language runtimes to improve performance by specializing frequently executed code paths. Unlike ahead-of-time compilation, JIT optimizers leverage runtime profiling information to make speculative assumptions about program behavior. These assumptions commonly involve execution hotness, type stability, and control flow predictability.

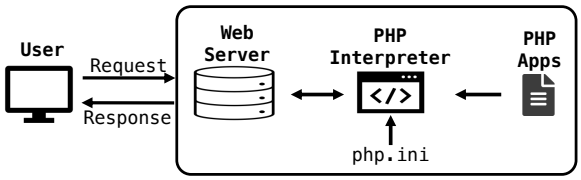


Fig. 1: Workflow of a PHP Application. Misconfiguration of the web server can cause request-routing and access-control flaws, and errors in the PHP apps can cause application-level vulnerabilities (e.g., injection or cross-site scripting). This work targets memory-safety defects inside the interpreter, focusing on a JIT-enabled PHP interpreter.

Modern JIT compilers often work in multiple stages. During interpretation, the runtime collects profiling information such as branch frequencies, variable types, and call targets. Once a region of code exceeds a hotness threshold, the JIT generates optimized native code specialized to the observed execution patterns. To preserve correctness, optimized code includes guards that validate assumptions at runtime; if a guard fails, execution deoptimizes and falls back to the interpreter.

Importantly, optimization is not applied to arbitrary programs. JIT engagement requires specific structural and profile conditions to hold, including sufficient execution frequency, stable types across profiling runs, and well-structured control flow. Programs that fail to satisfy these invariants remain in interpreted execution. As a result, optimized code paths are exercised only when these conditions are met. These conditions are independent of whether a program is otherwise well-formed.

PHP Deployment. In a typical PHP workflow (Figure 1), a browser request first arrives at a front-end web server, which serves static content directly and hands off dynamic content to the interpreter. Under the common FPM (FastCGI Process Manager) configuration [1], a supervising process maintains a pool of pre-forked worker processes. The web server dispatches the request over FastCGI to an available worker, which locates the corresponding script, compiles it to opcodes, executes them, and returns the rendered response. Each layer of this pipeline contributes its own attack surface. Misconfiguration of the front-end server and application-level flaws such as injection or cross-site scripting are orthogonal to our concerns; this work targets memory-safety defects inside the interpreter itself, and specifically inside the JIT optimizer [3].

III. THREAT MODEL AND SCOPE

Goal and Intended Use. JOTFUZZ is a *vulnerability-discovery* system. Its intended users are PHP developers and security researchers who proactively test the JIT optimizer for vulnerabilities before they can affect deployed systems. Accordingly,

operating JOTFUZZ does not require access to a victim server or any attacker capability. Generated programs are executed locally against an instrumented PHP build. JOTFUZZ identifies bug-triggering programs; constructing weaponized exploits is outside the scope of this work.

Deployment Model and Adversary. We assess the security relevance of discovered vulnerabilities in PHP deployments where the JIT is explicitly enabled and application execution can be influenced by an adversary. Following the two-scenario framing of prior PHP interpreter-exploitation work [3], we distinguish two forms of influence that differ in the attacker's ability to shape the executed code, while sharing the same ultimate goal of arbitrary command execution on the target. First, a **data-influence** adversary controls only values supplied to existing PHP code, mirroring the remote code execution scenario in which an attacker cannot deploy new scripts and can only craft requests that reach legitimate application code paths. This commonly occurs when request parameters, uploaded content, or other external data reach application and interpreter operations, and historical PHP vulnerabilities demonstrate that such attacker-controlled values can reach memory-safety-sensitive interpreter paths [4]. Second, a stronger **code-influence** adversary can affect the PHP code or control flow structure being executed, mirroring the sandbox-escape scenario in which an attacker can deploy arbitrary PHP within a restricted environment. This arises commonly in cloud computing environments where service providers deploy servers running PHP interpreters and allow users to upload PHP scripts for personal websites or remotely callable cloud functions. The PHP interpreter provides sandboxing capabilities for such scenarios which allow servers to restrict hardware usage, file access permissions, and access to sensitive built-in PHP functions.

Reaching JIT-Compiled Code. Because JOTFUZZ targets optimized execution, a vulnerable code region must be compiled by the JIT. Our evaluation uses PHP's tracing JIT (`opcache.jit=tracing`), under which frequently executed loops, functions, and traces become candidates for compilation. An attacker-influenced region can therefore reach optimized execution when it is exercised sufficiently often, for example through a hot loop over attacker-controlled data, repeated invocation of a request handler, or attacker-influenced code containing repeated calls. These JIT-activation conditions constrain the structure and execution profile of the program, rather than only the values of attacker-controlled inputs; §IV explains how JOTFUZZ models and satisfies these conditions during fuzzing.

Security Impact and Scope. We focus on memory-safety violations arising during JIT optimization or execution, including out-of-bounds accesses, use-after-free, and related memory-corruption primitives. Such faults have been demonstrated to enable stronger compromise such as sandbox escape and subsequent arbitrary code execution [5]. JOTFUZZ does not attempt to demonstrate end-to-end exploitation. Memory-safe semantic miscompilations are outside our current scope because they require a differential correctness oracle. We also exclude resource-exhaustion conditions caused solely by synthesized hot loops. Finally, our evaluation targets the official Zend implementation of PHP with tracing JIT enabled; other PHP runtimes and JIT configurations are outside the empirically evaluated scope.

IV. OVERVIEW

This section gives the overview of JOTFUZZ, our PHP JIT fuzzer which generates seeds using JOTs. We first identify the challenges in creating a JIT-targeting interpreter fuzzer. These challenges are addressed by our key insights which are actualized into our fuzzer, JOTFUZZ.

A. Challenges and Insights

JIT optimization plays a critical role in the efficiency of the PHP interpreter. Yet, current fuzzers fail to generate seeds that reliably exercise it. Developing a practical fuzzer which targets the JIT compiler raises two primary challenges, each of which motivates JOTFUZZ's design insights.

C1: Executable Programs. Generating a program that structurally satisfies a JIT precondition is insufficient if the program crashes before the optimizer engages. PHP scripts with undefined references raise fatal errors, terminating execution before any code becomes hot. A fuzzer that splices code fragments from disparate sources must therefore reconstruct each fragment's def-use context to produce executable programs.

I1: Dependency Slices. JOTFUZZ precomputes, for every statement in its corpus, the transitive closure of statements required to define the variables, functions, and types it references. Filling a template slot emits the entire slice, so fragments arrive with their def-use context intact.

C2: Optimizer Activation. JIT optimization only occurs when preconditions are adequately satisfied. Hotness thresholds, structured control flow, and stable data operations must hold before the optimizer engages (conditions that a valid program may never satisfy via Listing 1). Existing fuzzers do not encode these conditions in generated programs, so activation occurs only when a hot region arises coincidentally, and exploration of the optimizer is correspondingly unreliable and non-deterministic. This is the stage our design targets.

I2: JIT Optimization Constraints. Activation depends on program shape and execution profile rather than on syntax. We capture these in JOTs: syntax-agnostic constraint trees over control regions, trip counts, call multiplicity, and type stability. Because every generated program is generated to satisfy a JOT, activation is guaranteed. Furthermore, because JOTs are syntax-agnostic, they can be extracted from a compatible language's optimizer test suite. We mine V8's, whose tests are explicitly written to force optimization and which supplies an order of magnitude more templates than PHP's own.

Why Not Symbolic or Concolic Execution? The constraints that gate JIT optimization differ from the path predicates targeted by symbolic or concolic execution. Traditional constraint solving assumes a fixed program and searches for input values that drive execution along a desired path. Here,

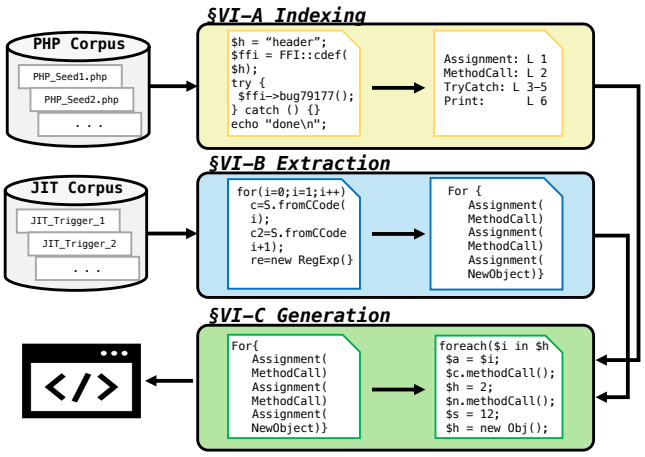


Fig. 2: Overview of JOTFUZZ

the program itself must be generated. Program control flow shape, execution frequency, call structure, and type behavior must jointly satisfy the optimizer's activation preconditions. These are therefore program-structure and execution-profile constraints, rather than predicates over program inputs.

B. System Overview

Figure 2 presents the architecture of JOTFUZZ, our fuzzer which directly targets the PHP JIT optimizer. JOTFUZZ consists of three modules:

1) **Indexing (§VI-A).** JOTFUZZ parses the entire PHP test suite into statement-level dependency graphs, computing for each statement the transitive closure of definitions required to execute it. Statements are indexed by their corresponding type for retrieval.
2) **Extraction (§VI-B).** JOTFUZZ analyzes JavaScript engine tests and PHP JIT tests, producing syntax-agnostic templates that encode the structural preconditions for JIT activation.
3) **Generation (§VI-C).** JOTFUZZ walks each JOT top-down, filling its slots with dependency slices drawn from the indexed corpus. The resulting program is executed on a sanitizer-enabled interpreter for bug detection.

V. JIT OPTIMIZATION CONSTRAINTS

JOTFUZZ extracts JOTs from the ASTs of JavaScript and PHP programs in engine test suites. These suites are written specifically to drive engines into optimized states, which makes them a dense source of the structural material stage two requires. Extraction retains only the optimization requirements: control flow shape, loop trip counts, call multiplicity, and type stability while discarding all syntax-related elements.

Control Tags. A `ControlTag` represents a region of structured control flow. Each control region has a *tag type* corresponding to the control flow construct it represents (e.g., `if`, `while`, `for`). A `ControlTag` always appears as the root element of a subtree, establishing the region type for its children. Loop control tags additionally carry two annotations: a *trip count* $n \in \mathbb{N} \cup \{\bot\}$ recording the number of iterations the loop performs, and a *loop type-stability* flag $s \in \{\text{stable}, \text{unstable}, \bot\}$ indicating whether the loop's carried variables retain their types across the back-edge.

Data Tags. A `DataTag` represents an instruction-level computation. Each `DataTag` has a *tag type* corresponding to the category of data operation it represents; for example, a `DataTag`'s type can be `assign`, `var_dec`, `func_call`, etc. Every `DataTag` is annotated with the *value type* $\tau \in \mathcal{T}$ it produces, drawn from the set:

$$\mathcal{T} = \{\texttt{int}, \texttt{float}, \texttt{string}, \texttt{bool}, \texttt{null}, \texttt{undefined}, \texttt{array}, \texttt{object}, \texttt{function}, \top\},$$

where $\top$ (`mixed`) denotes an unknown or non-monomorphic type. `DataTags` that write a variable (`assign`, `var_dec`, `update`) additionally carry a *stability* flag $\sigma \in \{\text{stable}, \text{unstable}, \bot\}$: the write is *stable* if it leaves the variable's type unchanged and *unstable* if it transitions the variable to a new type. Call `DataTags` (`func_call`) carry a *callee identity* κ (a label for the invoked function, or $\bot$ when undeterminable) and a *multiplicity* flag $\rho \in \{\text{true}, \text{false}, \bot\}$ that records whether the same callee is invoked more than once within the JOT.

JOT Formal Definition. A JOT is a tree whose first element is `ControlTag(main)`, followed by a sequence of elements, each of which is either a `DataTag` or a nested sublist (region) whose root is a `ControlTag`. Writing α for a `DataTag`'s annotation record $\langle \tau, \sigma, \kappa, \rho \rangle$ and β for a `ControlTag`'s annotation record $\langle s, n \rangle$, we have formally:

$$\begin{aligned}
\text{JOT} &::= [\texttt{ControlTag(main,}\beta\texttt{)}, E_1, \ldots, E_n] \\
E_i &::= \texttt{DataTag(}d,\alpha\texttt{)} \mid [\texttt{ControlTag(}c,\beta\texttt{)}, E_1, \ldots, E_m] \\
d &\in \{\texttt{assign}, \texttt{var_dec}, \texttt{func_call}, \ldots\} \\
c &\in \{\texttt{if}, \texttt{for}, \texttt{func}, \texttt{class}, \ldots\} \\
\alpha &= \langle \tau \in \mathcal{T},\ \sigma,\ \kappa,\ \rho \rangle \qquad \beta = \langle s,\ n \in \mathbb{N} \cup \{\bot\} \rangle
\end{aligned}$$

Annotations are optional. A component whose annotation cannot be resolved defaults to $\top$ or $\bot$, so the structural skeleton is always well-defined even when semantic inference is incomplete.

Mapping to JIT Preconditions. Each structural element and annotation in a JOT tree maps to a concrete JIT precondition.

Loop hotness. `ControlTag(for)` and `ControlTag(while)` produce loops that generate execution hotness. The trip-count annotation n determines whether the loop crosses the engine's hot-loop (on-stack-replacement) threshold and is therefore compiled at all.

Function hotness. `ControlTag(func)` produces function definitions that become candidates for optimization when called repeatedly. The multiplicity flag ρ marks those call sites that drive a callee past the hot-function threshold.

Type specialization and deoptimization. The value-type and stability annotations exercise the optimizer's type-specialization logic. A type-stable loop (s = stable) permits

Listing 2: JIT-Triggering Example

```
function f_0(x) {
    let s = 0;
    for (let i = 0; i < 100; i++) {
      s = s + (x|0) * i;}
    return s;}
f_0(2);
f_0(3);
```

Listing 3: JOT Example

```
[CT0('main'),
[CT1('func'),
  DT0('var_dec', type='int', stable=True),
  [CT2('for', type_stable=True, trip_count=100),
    DT1('assign', type='int', stable=True)],
  DT2('return', type='int')],
DT3('func_call', callee=CT1),
DT4('func_call', callee=CT1)]
```

Fig. 3: JIT-triggering code and its JOT representation.

the JIT to emit and retain specialized native code, whereas an unstable loop or an unstable write forces deoptimizations.

VI. JotFuzz

This section presents the design of JotFuzz. We explain the dependency-aware corpus indexing procedure (§VI-A), describe how JOTs are extracted from JavaScript seeds (§VI-B), and detail the template-guided generation algorithm (§VI-C).

A. Dependency-Aware Indexing

To fill JOT `ControlTags` and `DataTags` with real PHP source fragments, JotFuzz uses a corpus of PHP seed programs. Programs are analyzed and parsed into independent statements and labeled by their corresponding JOT type. This section describes how the corpus is analyzed and indexed.

Seed Corpus. Regression test suites are widely used as initial input corpuses [6], [2]. The PHP development team maintains a suite of over 20k scripts to ensure interpreter functionality. Each seed is a standalone PHP program containing a variety of operations (assignments, function calls, class definitions, etc.) representative of real-world PHP usage patterns.

Def-Use Analysis. Each seed file is parsed into an AST. The AST is then flattened. Statements nested inside control flow constructs (e.g, `if`, `for`) and scope boundaries (e.g., `function`, `class`) are turned into a linear sequence. Each control flow or scope construct is replaced by a *region header*, a node carrying only the construct's header expressions (conditions, loop variables), followed by the recursively flattened body statements. This linearization enables a single-pass dependency analysis while preserving the nesting context of each statement via metadata.

For each statement, JotFuzz computes two sets:

- **Defs**: Variables written by the statement. This includes assignment targets (`$a = ...`), destructuring assignments (`list($a, $b) = ...`), loop variables (`foreach` value/key), increment/decrement targets, etc.
- **Uses**: Variables read by the statement, excluding the left-hand side of simple assignments. For compound assignments (`$a += $b`), the left-hand side is included as both a def and a use.

Dependency Graph Construction. JotFuzz builds a statement-level dependency graph in a single pass over the flattened statement sequence. The algorithm maintains a last-definition map $L : Variable \rightarrow Stmt_ID$ that records, for each variable, the most recent statement that defined it. For each statement s_i with uses U_i:

$$dependency(s_i) = \{L[v] \mid v \in U_i \wedge v \in \text{dom}(L)\}$$

After computing dependencies, L is updated with s_i's definitions. In addition to data flow dependencies, the system computes *structural dependencies*. If s_i references a function, class, trait, or interface by name, it depends on the statement that defines that entity.

Transitive Closure and Dependency Slices. Given a target statement s_t, the *dependency closure* is the set of all statements transitively reachable through the dependency relation:

$$closure(s_t) = \{s_t\} \cup \bigcup_{s_j \in \text{depends_on}(s_t)} \text{closure}(s_j)$$

The *dependency slice* for s_t is the concatenation of source text for all statements in the closure, ordered by statement index. This slice is self-contained as it includes all variable definitions and structural declarations needed to execute s_t.

Type-Indexed Corpus. Each statement in the flattened corpus is classified along two axes. The first is its AST node type, which determines the JOT `DataTag` it can satisfy. The second is its inferred *value type*, drawn from the same lattice used to annotate JOTs. JotFuzz infers this type with a conservative bottom-up pass over the statement's AST. Literals, casts, and `new` expressions yield their obvious type; arithmetic, concatenation, and comparison operators propagate types through their operands; and a table of common built-in functions supplies return types (e.g., `strlen` → `int`). Any statement whose type cannot be determined statically degrades to `mixed`. The corpus index maps each node type to a list of *entries*; each entry records the statement's self-contained dependency slice together with its inferred value type, so that a data slot demanding a concrete type can be filled from a type-compatible statement.

B. JOT *Extraction*

JOTFUZZ extracts JOT constraints by analyzing the abstract syntax trees (ASTs) of JavaScript programs from engine test suites. JavaScript engine test suites (e.g., `mjsunit`) contain tests which are specifically designed to exercise JIT compilation and optimization behaviors.

Extraction proceeds in two steps: a semantic *analysis pass* that annotates the AST with type, stability, and trip-count information, followed by a structural *walk* that projects the annotated AST into a JOT tree.

Structural walk. For each JavaScript source file, JOTFUZZ parses the file into an AST and recursively walks the tree. The walker maintains a separate tree structure which is populated with control-tag and data-tag translations of each AST node type:

- **Control flow nodes** (`IfStatement`, `ForStatement`, `WhileStatement`, etc.) create a new nested list with the appropriate `ControlTag` and recursively walk their body.
- **Data flow nodes** (`AssignmentExpression`, `CallExpression`, `ReturnStatement`, etc.) append a `DataTag` leaf to the current environment.

This abstraction discards language and instantiation-specific surface details (variable names, literal values, operator choices) while preserving the structural skeleton, together with the semantic annotations, that determines whether a program will trigger optimization.

Type-stability inference. Because JavaScript and PHP are dynamically typed, type information is not present syntactically, thus JOTFUZZ infers it with a flow-sensitive abstract interpretation over the AST that runs before the structural walk and whose results are stamped onto the emitted tags. The analysis maintains an abstract environment mapping each in-scope variable to a lattice type in $\mathcal{T}$ and applies a transfer function to each expression. Literals map to their concrete types and arithmetic operators propagate numeric types. Bitwise operators yield `int` and `float` signals respectively. At control flow joins the environment is merged by the lattice's least-upper-bound, so a variable assigned incompatible types on different paths widens to $\top$. Function bodies are analyzed intraprocedurally with parameters bound to $\top$. The analysis is deliberately conservative. Any construct whose type it cannot establish (e.g., unresolved calls, dynamic property reads, closures) degrades to $\top$, and a write to a variable of unknown type is never reported as stable.

A variable write is labeled *stable* when it leaves the variable's abstract type unchanged and *unstable* when it transitions the type. For loops, JOTFUZZ additionally computes *loop-carried* type stability. JOTFUZZ evaluates the loop body's transfer function to a fixpoint and marks the loop *unstable* if any loop-carried variable's abstract type widens. Because the lattice is of bounded height, the fixpoint converges in a small number of passes. Unstable loops correspond directly to the deoptimization-inducing loops that stress the optimizer's guard machinery.

Algorithm 1: Template-Guided Generation

Input: JOT T; corpus index I; dependency cache D; thresholds θ_ℓ, θ_f
Output: PHP source program
State: H hoisted decls; Σ defined vars; K callee $\kappa \mapsto$ name
Function `Synthesize(`T`)`**:**
  $H, \Sigma, K \leftarrow \emptyset$; $B \leftarrow$ `EmitRegion(`T`, 0)`
  return `"<?php"` $\|$ `Dedup(`H`)` $\|$ B
Function `EmitRegion(`R`, `δ`)`**:**
  $c \leftarrow R[0]$ `// ControlTag,` $\beta = \langle s, n \rangle$
  if c *is a loop* **then** `// floor at hot-loop threshold`
    $L \leftarrow$ counted loop, bound $\max(\beta.n, \theta_\ell)$, var `Fresh(`$i$`)`
  else if $c =$ `func` **then**
    $K[\kappa(c)] \leftarrow$ `Fresh(`f`)`; $L \leftarrow$ decl. header for $K[\kappa(c)]$
  else $L \leftarrow$ header for c (`if`, `class`, `try`, ...)
  foreach $e \in R[1{:}]$ **do**
    $L \leftarrow L \| ($$e$ is a region ? `EmitRegion(`e`, `$\delta{+}1$`) : FillSlot(`e`, `δ`))`
  return $L \|$ close(c, δ)
Function `FillSlot(`e`, `δ`)`**:**
  `//` $e =$ `DataTag(`$d, \langle \tau, \sigma, \kappa, \rho \rangle$`)`
  if $d =$ `func_call` $\wedge\ \kappa \in \mathrm{dom}(K)$ **then return** $(\rho\,?\,\theta_f : 1)$ calls to $K[\kappa]$ `// hot callee`
  $C \leftarrow \{x \in I[d] : (\tau{=}\top \vee x.\tau{=}\tau) \wedge (\sigma{=}\bot \vee x.\sigma{=}\sigma)\}$
  if $C = \emptyset$ **then** $C \leftarrow I[d]$ `// fall back if over-constrained`
  $x \leftarrow$ `Sample(`C`)`; $(H', P) \leftarrow$ `SplitHoistable(Slice(`x`, `D`))`
  $H \leftarrow H \cup H'$; $\Sigma \leftarrow \Sigma \cup x.defs$
  return indent(`Join(`P`, `$x.free$`, `Σ`)`, δ) `// bridge free uses`

Trip-count inference. For counted loops with an integer induction variable, constant initial and bound values, and a constant step, JOTFUZZ solves for the exact iteration count analytically and records it as the trip-count annotation n. Data-dependent bounded loops (e.g., `for-in`) are left unresolved ($n = \bot$), reflecting that their iteration counts cannot be established statically.

Call-multiplicity labeling. Each `func_call` tag is first labeled during the walk with the syntactic identity κ of its callee (a bare function name such as `f`, or a member reference such as `obj.method`). After the walk completes, a whole-JOT post-pass counts the occurrences of each callee identity across the entire tree and sets the multiplicity flag ρ to true for every call whose callee is invoked more than once. This whole-tree view is essential as a function becoming hot is a global property of the program, not of any single call site.

Listing 2 presents an example JavaScript JIT-triggering example where the `for` loop in function `f_0` causes the code to get hot and thus trigger optimization. Listing 3 illustrates the corresponding JOT, where each element corresponds to its JavaScript statement and carries the inferred annotations. The loop tag records its trip count and type-stability, each write records its value type and stability, and the repeated calls to `f_0` are marked with multiplicity.

C. Template-Guided Program Generation

Given a JOT and the indexed corpus, JOTFUZZ generates a PHP program by walking the constraint top-down and filling each component with concrete PHP source. Algorithm 1 presents the generation algorithm.

Control Region Generation. To satisfy stage two, each emitted loop must actually cross PHP's threshold. JOTFUZZ

sets the iteration count to $\max(n, \theta_\ell)$, where n is the JOT's recorded trip count and θ_ℓ is PHP's hot-loop threshold. The recorded count is thus a lower bound re-targeted to the destination engine rather than transferred literally, thus a JavaScript loop that crossed V8's threshold at 50 iterations is emitted with enough iterations to cross PHP's. Call sites are re-targeted identically, with a multiplicity-flagged callee invoked θ_f times. A global counter generates unique names (`f_0`, `f_1`, `C_0`, `i_0`, etc.) to avoid collisions across regions.

Listing 4 exemplifies control region generation based off the JOT in Listing 3 with each line color-coded to its corresponding JOT line. Line 2 fills the `ControlTag` for the function definition while Line 5 nests a `for` loop inside the function definition as defined by the JOT. This matching ensures that the final PHP output faithfully reflects the control-flow behavior of the constraints.

Data Slot Filling. Similar to control region generation, each `DataTag` is filled by querying the corpus index for statements matching the tag's node type. When the `DataTag` additionally pins a concrete value type, JoTFuzz restricts the candidate pool to entries whose inferred value type matches, so that the created statement exercises the same type the JOT observed in the seed. If the JOT is type-agnostic (`mixed`) or no type-compatible candidate exists, JoTFuzz falls back to the full pool. A candidate statement is then selected uniformly at random from the eligible matches. The system retrieves the statement's dependency closure from the pre-computed cache and constructs the *dependency slice*.

The dependency slice is partitioned into two categories:

- **Hoistable definitions**: Function, class, trait, and interface declarations that must appear at the top level of the PHP script (outside control structures) to be valid.
- **Inline statements**: All other statements, which are placed directly at the data slot's position in the synthesized output.

In Listing 4, JoTFuzz fills each `DataTag` with a data dependency slice from the seed corpus. Notably, multiple statements correspond to one tag. For example, Line 3 of the JOT (Listing 3) calls for a variable declaration and corresponds to Lines 3 and 4 in the emitted PHP source code. Line 4 of Listing 4 ($\$counter = count(\$items);$) fills the variable declaration constraint, but the dependency slice includes Line 3 ($\$items = [1, 2, 3, 4];$. This illustrates the utility of dependency slices. By using both lines to satisfy the constraint, no semantic error is introduced.

Variable Join Points. JoTFuzz introduces *variable join points* between adjacent data slots to create cross-slot data-flow connections. Given the defs of a preceding slot and the free uses (variables used but not defined within the dependency closure) of the current slot, JoTFuzz creates a join variable that bridges the two. This mechanism increases the data flow complexity of generated programs.

VII. Implementation

To evaluate the effectiveness of our approach, we implement JoTFuzz which fuzzes the PHP interpreter's optimizer using constraint-guided seed synthesis. JoTFuzz is a black-box fuzzer implemented in Python and shell script, consisting of approximately 4,500 lines of code (LoC).

Listing 4: PHP seed produced from JOT

```
1  <?php
2  function f_0($a) {
3      $items = [1, 2, 3, 4];
4      $counter = count($items);
5      for ($i_0 = 0; $i_0 < 100; $i_0++) {
6          $path = __DIR__ . "/data.txt";
7          $raw = file_get_contents($path);
8          $name = trim($raw);
9          $output[] = strtolower($name);
10     }
11     $payload = ["ok" => true];
12     $result = json_encode($payload);
13     return $result;
14 }
15 $haystack = "abc123";
16 var_dump(preg_match('/\d+/', $haystack));
17 $x1 = f_0(1);
18 $x2 = f_0(1);
```

JOTs are extracted from a corpus of JavaScript tests [7] and PHP optimizer test cases. JavaScript test cases provide 5,698 JOTs, while 475 JOTs were mined from the PHP test suite. The official PHP test suite [8], which contains regression tests and previous bugs, is used to source dependency slices. The extraction module uses the acorn parser [9] to convert JavaScript into ASTs which are then analyzed to generate JOTs. The indexing module leverages PHP-Parser [10] to convert PHP source code into an AST which is then analyzed to index dependency slices.

The PHP interpreter is compiled with Clang's Address-Sanitizer (ASan) and UndefinedBehaviorSanitizer(UBSan) to detect memory safety violations and undefined behaviors. Both JOTs and dependency slices are serialized and stored as *pickle* files [11], allowing both to be reused during fuzzing without redundant analysis.

VIII. Evaluation

- **RQ1:** Can JoTFuzz discover new bugs in the PHP JIT optimizer?
- **RQ2:** How do JOTs contribute to the overall performance of JoTFuzz? How much do dependency slices contribute?
- **RQ3:** How does JoTFuzz compare to state-of-the-art fuzzers?
- **RQ4:** How effective is JoTFuzz at generating valid and executable PHP seeds? How many seeds trigger JIT compilation?
- **RQ5:** How efficient is JoTFuzz's seed generation?

A. Experimental Setup

We deployed JOTFUZZ on a dedicated server running Ubuntu 22.04 with 16 CPU cores and 180 GB of RAM. We fuzzed the official PHP interpreter (`v8.5.0-dev`), compiled daily from source with sanitizers. Following prior work, we initialized all fuzzing campaigns using the official PHP test suite [8], which provides a comprehensive set of PHP programs.

Answering RQ1. To answer RQ1, we evaluated JOTFUZZ on `PHP-8.5.0-dev`, the snapshot built directly from the PHP master branch that is updated daily. The daily development release reflects the latest bug patches, thereby reducing the number of duplicate bug discoveries.

JOTFUZZ was run for a period of one month with both AddressSanitizer (ASan) and UndefinedBehaviorSanitizer (UBSan) enabled to detect both memory safety violations and undefined behavior during execution. All generated seeds were executed on the instrumented build with all sanitizer outputs being recorded for analysis.

Each bug was manually triaged to identify the affected interpreter component and determine the underlying root cause. We assess the practical impact of each bug in accordance with our defined threat model and determine its severity.

Answering RQ2. To answer RQ2, we perform an ablation study to quantify the efficacy of JOTFUZZ's design decisions. First, JOTs impose control flow structure and hotness on generated seeds, and we measure how much this structure contributes to coverage and bug discovery. Second, dependency slices reduce semantic errors by preventing undefined structural usages, thus we evaluate whether this semantic safety feature significantly bolsters the overall performance of JOTFUZZ. Finally, we evaluate whether cross-language structure transfer contributes to system performance. PHP's own JIT test suite is a comparatively small source of optimizer-triggering structure, yielding only 475 JOTs. JIT-triggering structure is a syntax-agnostic property, and JavaScript engine test suites are consequently a usable supply of additional templates for PHP. We therefore evaluate whether a larger corpus of JOTs via JavaScript-derived templates meaningfully contributes to JOTFUZZ's performance.

We construct four ablated variants of JOTFUZZ. (1) A *PHP-JOT* variant retains the full pipeline but draws templates only from PHP-sourced JOTs, ablating cross-language transfer while holding every other component fixed; this variant isolates the structural template thesis. (2) A *No-JOT* variant randomly splices PHP dependency slices, removing template guidance entirely. (3) A *No-Dependency Slice* variant splices PHP statements without dependency considerations. (4) A *Baseline* variant appends PHP statements together without JOTs or dependency slices.

Each configuration performs a 24-hour fuzzing campaign under identical conditions (limited to 8 cores), repeated for seven independent trials. For each campaign we measure line coverage across the full interpreter and its major subsystems.

Answering RQ3. To evaluate the effectiveness of JOTFUZZ against current state-of-the-art baselines, we conduct a comparative study with three PHP fuzzers. While general-purpose fuzzers rely on ground-truth benchmarks like Magma [12] and FuzzBench [13] for evaluation, no equivalent benchmark exists for PHP. Thus, evaluation is performed on real-world interpreter code without ground truth. We exclude generic byte-level fuzzers (AFL, AFL++) as ill-fitted for highly structured PHP code generation. Given these constraints, we select three state-of-the-art fuzzers which are publicly available, can generate syntactically valid PHP scripts, and are explicitly designed for structured fuzzing.

- **FlowFusion** [2], the official fuzzer used by the PHP development team, generates new seeds by merging data flows extracted from existing PHP test cases. We use FlowFusion exactly as recommended by its authors, including their provided configuration and workflow.
- **PolyGlot** [5], a BNF-based, generative fuzzer capable of producing semantically valid inputs through annotated grammar rules. We configure PolyGlot for PHP following its official documentation and use both the PHP grammar and seed corpus distributed with the project.
- **Domato** [14], a grammar-based fuzzer developed by Google Project Zero, includes a built-in PHP grammar. We use Domato's PHP grammar unchanged for a faithful comparison.

To assess coverage performance, we track coverage and JIT-bug discovery over 24-hour fuzzing campaigns for seven trials (limited to 8 cores).[1] JIT compilation is active for all tests (i.e., `opcache.jit=tracing`).

Answering RQ4. To answer RQ4, we evaluate the ability of JOTFUZZ and baseline fuzzers to generate *valid* PHP programs and their ability to trigger JIT compilation. A seed is only useful for JIT fuzzing if it satisfies two conditions in sequence: it must execute without a fatal error (validity), and its execution must actually engage the optimizer (JIT compilation). We therefore decompose each fuzzer's effectiveness into two measured rates and report their product, the *JIT-trigger rate* which is the fraction of all generated seeds that both run to completion and trigger JIT compilation.

We generate 1,000 PHP programs per fuzzer and execute each seed on an uninstrumented PHP interpreter. A seed is considered *valid* if the interpreter exits with code 0 and produces no fatal diagnostics (parse errors, uncaught exceptions, or fatal runtime errors). This definition captures programs that successfully reach the execution stage where JIT compilation could occur. Furthermore, we use PHP's builtin JIT status functionality to determine if JIT compilation occurred (i.e., `opcache_get_status`).

Answering RQ5. To answer RQ5, we examine the throughput of JOTFUZZ and baseline fuzzers. Throughput is a critical measurement for fuzzers as higher seed generation and execution increases the likelihood of bug discovery. JOTFUZZ's throughput is primarily determined by the efficiency of its

[1] v8.5-dev, Commit Hash: 3f3a266a2bab9fda95096cf4a17156a5dc66e8bb

constraint-guided seed synthesis. This analysis investigates whether the algorithm and implementation serve as an efficient fuzzing technique compared to state-of-the-art baselines. We assess throughput by recording the number of seed executions performed over seven 24-hour periods.

Bug Deduplication. Bugs are deduplicated by sanitizer stack traces. Each bug requires a unique input and receives a different issue tracker from the development team.

Coverage Oracle. Code coverage is a standard metric for evaluating the effectiveness of fuzzing techniques [15]. To measure coverage, we use `gcovr` [16] following the recommended instructions provided by its developers. Coverage is collected at the line level across the entire PHP interpreter as well as within major subsystems.

Reproducibility and Statistical Rigor. For reproducibility, we release an artifact containing: (i) the full source code of JOTFUZZ, (ii) the generated JOTs, and (iii) the analyzed dependency slices needed to reproduce experimental results.[2] To ensure the statistical significance of our results, we followed the best practices outlined by Klees et al. [15]. To account for the stochastic nature of fuzzing, we performed seven independent runs for each ablation, comparison, validity, and throughput experiment (RQ2-5). We used the two-sided Mann-Whitney U test to evaluate whether the observed differences were statistically significant ($p < 0.05$) and report effect size alongside p-values with Vargha–Delaney $\hat{A}_{12}$.

B. Results

1) Vulnerability and Bug Discovery: The effectiveness of JOTFUZZ is demonstrated practically through its ability to find JIT bugs in the latest PHP interpreter. Over a one-month fuzzing campaign on the daily updated `PHP-8.5.0-dev`, JOTFUZZ discovered **20 confirmed bugs**. Of these, **14 were JIT-related** and were acknowledged by the PHP development team. Further analysis showed that 18 of the 20 bugs are security relevant under our code-influence threat model.

Distribution. Table I lists the set of confirmed bugs. The *Component* column indicates the affected subsystem, the *Class* column displays the corresponding common weakness enumeration (CWE), the *Description* column includes a short description of the bug, and the *Status* column indicates whether the bug is fixed (*Fx*), verified (*Vd*), or a duplicate (*Dp*). The *Fix* column reports the number of lines of code changed to fix the bug, and each bug is marked to indicate if it is security relevant under our threat model. We further label each JIT bug to indicate whether it was derived from a PHP or JavaScript JOT.

Because of the JIT-targeting design of JOTFUZZ, the discovered bugs are composed primarily of memory errors affecting PHP's JIT optimizer. More specifically, the discovered bugs corresponded to **6 distinct CWEs**[3]. The most common vulnerability classes were:

- **CWE-416** (Use-After-Free; 6): the optimizer retains references to memory freed during a transformation pass, so subsequent accesses read or write a dangling pointer. Attacker-controlled reallocation of the freed region is a well-established path to memory corruption and arbitrary code execution.
- **CWE-121** (Stack-based Buffer Overflow; 5): writes past the bounds of a stack-allocated buffer in the interpreter's hash, variable, and object handling routines. Adjacent stack data, including saved registers and return addresses, may be overwritten, making control-flow hijack feasible.
- **CWE-122** (Heap Buffer Overflow; 4): out-of-bounds heap writes arising from incorrect size or index computation during optimization. These corrupt adjacent heap metadata or object fields and can escalate to code execution.

The discovered bugs span 7 different interpreter subsystems with the majority (14) affecting the JIT optimizer, indicating that while JOTFUZZ can uncover bugs in various components, it primarily targets PHP's JIT subsystem.

Security Relevance. Under the defined threat model (§III), we assess 18 of the 20 discovered bugs as security relevant under our code-influence adversary, including 13 of the 14 JIT bugs. We classify a finding as security relevant when attacker-controlled PHP execution can trigger either a memory-safety violation or a meaningful availability impact. Heap and stack memory corruptions (CWE-121, CWE-122, and CWE-416), including buffer overflows and use-after-free, constitute well-established primitives that have historically enabled stronger compromise such as arbitrary code execution [3]. NULL-pointer dereferences (CWE-476) are also security relevant because they allow attacker-controlled PHP code to terminate the interpreter process, resulting in denial of service. We do not, however, claim that these individual findings are directly exploitable for code execution, as we do not demonstrate end-to-end exploitation. We exclude the remaining two bugs from our security-relevant set: although integer overflows (CWE-190) can contribute to exploitation when chained with subsequent memory-safety violations, we do not demonstrate such a chain and therefore conservatively treat the observed overflow as a correctness issue; we similarly classify the unreachable program point (CWE-561) as a correctness bug.

Security-Relevant Bug Analysis: Phi Nodes. JOTFUZZ uncovered a UAF vulnerability in the JIT compiler (presented in Listing 5). PHP's JIT builds a single static assignment (SSA) form in which phi nodes reconcile values that flow together at control flow join points. To uncover this bug, JOTFUZZ placed repeated, interdependent assignments inside a hot loop, forcing the JIT to construct phi nodes at the loop's back-edge and then invoke the pass that prunes redundant

[2] https://anonymous.4open.science/r/jotfuzz-1063/

[3] The bugs found by JOTFUZZ correspond to the following CWEs: CWE-190 Integer Overflow or Wraparound, CWE-476 Null Pointer Dereference, CWE-416 Use-after-Free, CWE-561 Unreachable Program Point, CWE-122 Heap-based Buffer Overflow, CWE-121 Stack-based Buffer Overflow [17]

TABLE I: Bugs confirmed by the PHP development team. JOTFUZZ disclosed 20 bugs in PHP `8.5.0-dev`. Issue IDs are redacted for anonymized review.

#	Issue ID	Component	Class	Description	Status	Fix
1‡ ❖	[redacted]	JIT	CWE-476	Member access within null pointer (segfault) in zend_builtin_functions.c	Dp	
2†❖	[redacted]	JIT	CWE-122	Heap buffer overflow in ir_sccp.c	Fx	+72 -5
3‡ ❖	[redacted]	JIT	CWE-476	Member access within null pointer (segfault) in ir_gm.c	Fx	+471 -3
4†❖	[redacted]	JIT	CWE-122	Heap buffer overflow in zend_cfg.c	Vd	
5❖	[redacted]	JIT	CWE-190	Integer overflow in zend_inference.c	Vd	
6†❖	[redacted]	JIT	CWE-122	Incorrect SCCP handling for arrays causes heap overflow in optimizer	Fx	+25 -2
7†❖	[redacted]	JIT	CWE-416	Heap use-after-free in ir_private.h	Fx	+1 -0
8†❖	[redacted]	JIT	CWE-416	Heap use-after-free in zend_vm_execute.h	Vd	
9†❖	[redacted]	JIT	CWE-122	Heap buffer overflow in zend_cfg.c	Vd	
10†❖	[redacted]	JIT	CWE-416	Heap use-after-free in zend_execute.h	Dp	
11†*	[redacted]	JIT	CWE-121	Stack-based buffer overflow in zend_variables.c	Vd	
12†*	[redacted]	JIT	CWE-121	Stack-based buffer overflow in zend_hash.c	Fx	+52 -1
13†*	[redacted]	JIT	CWE-121	Stack-based buffer overflow in zend_objects.c	Vd	
14†*	[redacted]	JIT	CWE-121	Stack-based buffer overflow in zend_hash_compare	Vd	
15†	[redacted]	streams	CWE-416	Heap use-after-free in streams.c	Fx	+15 -0
16†	[redacted]	test	CWE-416	Heap use-after-free in test.c	Vd	
17‡	[redacted]	snmp	CWE-476	Member access within null pointer in snmp.c	Fx	+72 -8
18	[redacted]	streams	CWE-561	Unreachable program point in plain_wrapper.c	Vd	
19†	[redacted]	streams	CWE-416	Use-after-free in streams module	Dp	
20†	[redacted]	standard	CWE-121	Stack-based buffer overflow in standard extensions	Vd	

† RCE primitive under §III. ‡ DoS primitive under §III. ❖ JavaScript-derived JOT. * PHP-derived JOT.

ones. Developers stated, "*When removing phis from the live inputs, the life_inputs is freed. However, a next iteration may again need to store life_inputs, but this is now a dangling pointer even though it should allocate a new pointer.*" The bug is a direct consequence of JOTFUZZ's design as only a program that both drives the loop hot and populates it with enough live values to require phi reconciliation reaches the vulnerable pruning logic, a structural precondition that conventional fuzzers rarely satisfy.

Security-Relevant Bug Analysis: SCCP. PHP's optimizer includes an optimization pass called SCCP (Sparse Conditional Constant Propagation) that performs constant propagation and dead code elimination on compiled opcodes. As triggered by Listing 6, a flaw in SCCP caused incorrect handling of arrays containing partial objects. JOTFUZZ triggers this issue by constructing arrays that include partially initialized objects. The PHP development team noted, "*In SCCP, arrays containing partial objects must be marked as partial so that their values are not accidentally propagated.*"[4]

JOTFUZZ uncovers security-relevant JIT bugs that require specific optimization conditions to trigger.

2) Ablation Study: Figure 4 presents the 24-hour code coverage trends for JOTFUZZ and its ablated variants across the PHP interpreter and its primary subsystems. Following best practices [15], we apply the two-sided Mann-Whitney U test ($\alpha = 0.05$) and report the Vargha–Delaney $\hat{A}_{12}$ effect size alongside each p-value in Table II. Every variant is separated from the full configuration at $p < 0.05$ in every subsystem.

Component contributions. Removing dependency slices is the most substantial ablation everywhere, costing 12.4 percentage points (pp) of full-system coverage (39.6% → 27.2%), 10.8 pp in the Zend engine, 9.6 pp in the main functions, and 12.3 pp in the modules. This ordering follows from the failure mode each component addresses. A seed that references an undefined variable, function, or class aborts early preventing higher coverages from being reached. Removing JOTs is less costly but still significant, at 5.2 pp full-system. The two components are at least additive with the *Baseline* variant, which ablates both, falling to 22.5% full-system. Structure without executability produces programs that die before the structure matters, and executability without structure produces programs that run but stay sequential.

Subsystem-specific effects. The relative weight of the two components varies with what each subsystem does. In the Zend engine, which executes opcodes and manages memory, ablating JOTs costs only 3.4 pp against the 10.8 pp cost of ablating dependency slices, reflecting that reaching the execution engine at all is primarily a validity problem. The gap is widest in the modules, where JOT ablation costs 2.9 pp

Listing 5: UAF from Phi Nodes

```
$o = new stdClass();
$data = "Testing openssl_sign()";
for ($i_0 = 0; $i_0 < 100; $i_0++) {
    $src_tc = imagecreate(19,19);
    $white = imagecolorallocate($src_tc
        ,255,255,255);
    $n = gc_collect_cycles();
    $test_string = str_repeat('Eins zwei drei',
        2000);
    $replaced = preg_replace('/\s/', '-',
        $test_string); }
$doubles = array(29000, 9999, (float)0);
foreach ($doubles as $d) {
    var_dump((string)$d); }
```

[4]The presented vulnerabilities have been fixed by the PHP developers. All detected bugs were responsibly disclosed (Ethical Considerations).

Listing 6: Overflow from SCCP Mishandling

```
function f_1() {
    $obj->a = 3; $blank_line = <<<EOT
    EOT;
    $array_default_key = array('three', '4');
    $dom = DOM\HTMLDocument::createFromString('<
        p>foo</p>', LIBXML_NOERROR);
    $test = function($a='a',$b='b',$c='c') {
        echo "a = $a, b = $b, c = $c\n";};
    $dom = new DOMDocument;
    $comment = $dom->createComment("foobar");
    for ($i_2 = 0; $i_2 < 100; $i_2++) {
        $obj = $reflector->newLazyProxy(function
            ($obj) {
            var_dump("initializer");
            return new C();});
        $tStamp=mktime(17,17,17,1,4099,1970);
        $x = "foo";
        $step = function () {
            throw new Exception();};
        $a = 30;
        $it = new RecursiveArrayIterator([[1]]);
        $objs = []; $obj = new stdClass;
        $objs[] = $obj;
        $table_name = "table_pg_fetch_object";
        $sql="SELECT * FROM $table_name WHERE
            num = 0";
        $pdo=PDOTest::test_factory(__DIR__ . '/n
            .phpt',Pdo\Pgsql::class,true);}
    return $input % 2 === 0; }
```

against 12.3 pp for slice ablation. Extension code is reached through concrete API calls drawn from the corpus, and the def-use context that makes those calls executable matters far more than the control-flow shape they are wrapped in. The main functions, which perform argument parsing and runtime setup, show the most balanced split (5.8 pp versus 9.6 pp).

JOT corpus expansion. The *PHP*-JOT variant isolates the contribution of JS-derived templates, drawing templates only from the 475 PHP-sourced JOTs rather than additionally from the 5,698 mined from V8's test suite. Ablating the JavaScript-derived templates costs 3.9 pp of full-system coverage (39.6% $\rightarrow$ 35.7%), 2.4 pp in the Zend engine, 3.3 pp in the main functions, and 1.5 pp in the modules. All four differences are statistically significant, though with the weakest effect sizes in the study ($\hat{A}_{12} = 0.8$ in the modules at $p = 0.040$).

Prior work establishes that a larger, more diverse seed corpus directly improves interpreter fuzzing performance [18]. Our findings are consistent with this precedent as a larger JOT corpus containing JavaScript-derived templates improving fuzzing performance. We further attribute template ancestry with the discovered vulnerabilities. Of the 14 JIT bugs, **10** were found by seeds matched to JavaScript-derived templates.

Results indicate the combination of dependency slices and JOTs greatly improves JOTFUZZ's performance.

3) Comparison Study: We compare JOTFUZZ against three state-of-the-art structured PHP fuzzers, FlowFusion, PolyGlot, and Domato. Figure 5 reports mean line coverage across seven independent 24-hour trials, and Figure 6 reports average JIT bug discovery.

Variant	No Deps	No JOTs	PHP-JOTs	Baseline	JOTFUZZ
Comp.	**Full**	-	-	-	-
Cov.	27.2	34.4	35.7	22.5	39.6
P-Value	0.002	0.002	0.002	0.002	-
$\hat{A}_{12}$	1.0	1.0	1.0	1.0	-
Comp.	**Zend**	-	-	-	-
Cov.	50.8	58.2	59.2	41.7	61.6
P-Value	0.002	0.002	0.002	0.002	-
$\hat{A}_{12}$	1.0	1.0	1.0	1.0	-
Comp.	**Main**	-	-	-	-
Cov.	45.0	48.8	51.3	36.3	54.6
P-Value	0.001	0.001	0.003	0.002	-
$\hat{A}_{12}$	1.0	1.0	0.9	1.0	-
Comp.	**Modules**	-	-	-	-
Cov.	24.2	33.6	35.0	18.8	36.5
P-Value	0.001	0.021	0.040	0.002	-
$\hat{A}_{12}$	1.0	0.9	0.8	1.0	-

TABLE II: P-values of ablation study

Fuzzer	FlowFusion [2]	PolyGlot [5]	Domato [14]	JOTFUZZ
Cov.	37.0%	25.1%	17.3%	**39.6%**
P-Value	0.1050	0.0020	0.0020	-
$\hat{A}_{12}$	0.7	1.0	1.0	-
Bugs	0.14	0	0	**1.14**
P-Value	0.0020	0.0006	0.0006	-
$\hat{A}_{12}$	0.9	1.0	1.0	-
JIT rate	1.8%	6.4%	1.5%	**42.1%**
P-Value	0.0020	0.0020	0.0020	-
$\hat{A}_{12}$	1.0	1.0	1.0	-
Thru.	47.0k	1,855.0k	25.7k	**230.4k**
P-Value	0.0020	0.0020	0.0020	-
$\hat{A}_{12}$	1.0	0	1.0	-

TABLE III: Comparison against State-of-the-art

Coverage. JOTFUZZ achieves the highest overall coverage at **39.6%** with FlowFusion close behind at 37.0%; PolyGlot reaches 25.1% and Domato 17.3%. The ordering is consistent with each tool's design. FlowFusion bootstraps from the same official test suite as JOTFUZZ and fuses data flows extracted from real programs, so it inherits the semantic richness of the corpus and exercises a broad portion of the interpreter. PolyGlot and Domato instead generate programs from grammars; without grounding in real def-use context, a large fraction of their output triggers fatal errors during compilation and never reaches deep interpreter state, capping the coverage they can accumulate. Under the two-sided Mann-Whitney U test, JOTFUZZ's advantage over PolyGlot and Domato is statistically significant ($p = 0.002$, $\hat{A}_{12} = 1.0$ in both cases), whereas the gap over FlowFusion is not ($p = 0.105$, $\hat{A}_{12} = 0.7$). This parity is expected as aggregate coverage rewards breadth across the whole interpreter, which a data flow approach over the same corpus attains directly, while JOTFUZZ is designed to concentrate execution inside optimizer passes. The separation between the two tools appears in the metrics that measure that objective (i.e., JIT-trigger rate and JIT bug discovery) rather than in total lines touched.

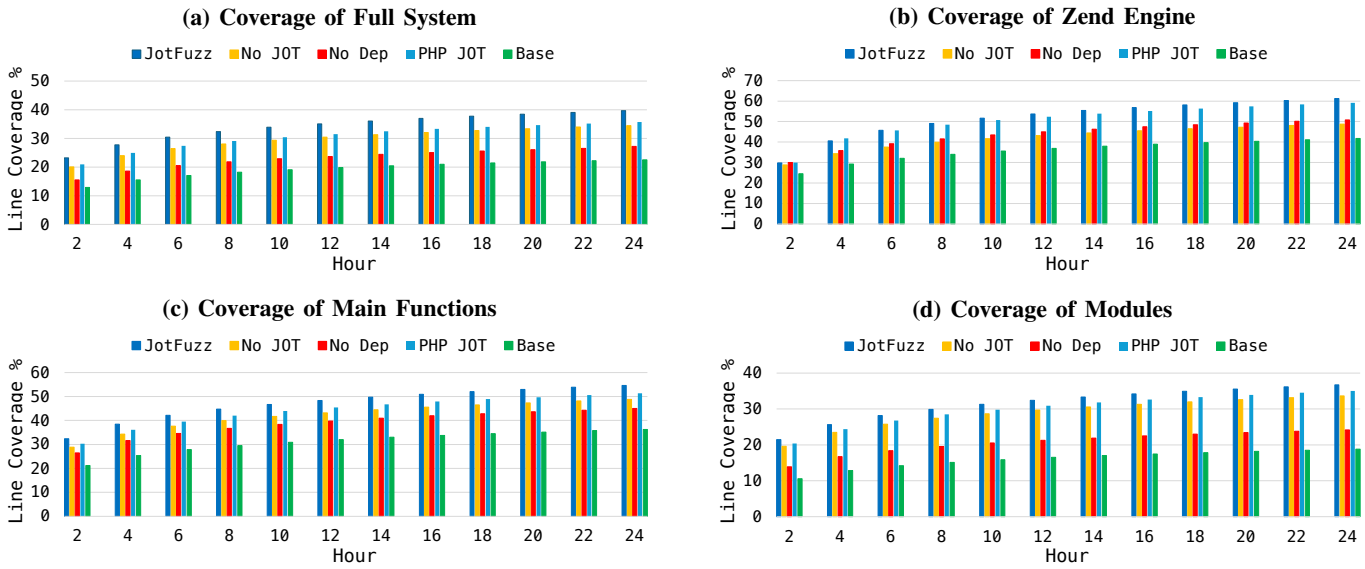


Fig. 4: Coverage Comparison of Ablated Variants vs JOTFUZZ.

Bug Discovery. Figure 6 shows the average JIT bug discovery over seven trials. JOTFUZZ averages **1.14** unique JIT bugs. FlowFusion averaged 0.14 while neither PolyGlot nor Domato discovered any bugs. This is the central result of the comparison: despite near-parity in aggregate coverage, JOTFUZZ finds substantially more optimizer bugs than FlowFusion, and the two remaining fuzzers find none at all. The gap is a direct consequence of JOTFUZZ's design. Each bug it uncovered required not merely reaching the optimizer but placing specific structure inside a hot region: the phi-node use-after-free of Listing 5 appears only when a loop is both driven hot and populated with enough live values to force phi reconciliation, and the SCCP array bug requires partially initialized objects inside a constant-propagated array. FlowFusion and the grammar fuzzers occasionally produce hot loops by chance, but they do not enforce hotness as an invariant, so they reach these transformation passes rarely and populate them with the required structure rarer still.

JOTFUZZ outperforms state-of-the-art baselines in JIT bug discovery. The constrained seed generation of JOTFUZZ enables targeted fuzzing of the JIT compiler.

4) Seed Validity and JIT-Trigger Rate: JIT-trigger rate instantiates the two-stage formulation directly. The validity rate measures stage one, the conditional trigger rate measures stage two, and their product is the fraction of generated seeds that are useful for JIT fuzzing at all.

JOTFUZZ produces valid seeds at a rate of **42.1%** (421.3 of 1,000 on average). Every valid JOTFUZZ seed triggers JIT compilation, so its validity rate and JIT-trigger rate coincide at **42.1%**. Every JOT contains at least one hotness-generating control region, so any JOTFUZZ program that runs at all necessarily crosses a hotness threshold. Validity is thus the only barrier between a generated program and optimized execution,

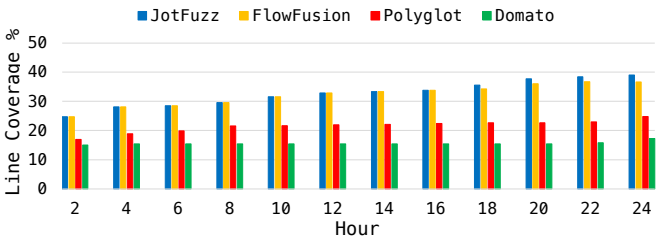


Fig. 5: Coverage Comparison against State of the Art

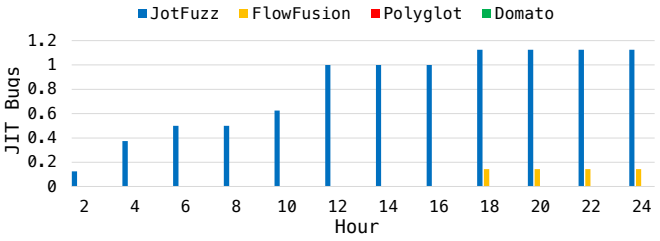


Fig. 6: Bug Discovery Comparison against State of the Art

which is what JOTFUZZ's dependency-slice construction is designed to remove.

The baselines exhibit the opposite pattern. Even when they generate valid programs, those programs rarely become hot. FlowFusion attains a validity rate of 2.8% with 64% of its valid seeds trigger compilation, for a JIT-trigger rate of 1.8%. FlowFusion's high coverage and low trigger rate are consistent. Though it exercises the interpreter broadly it lacks any hotness invariant and thus engages the optimizer only when a hot loop arises coincidentally. PolyGlot reaches 42.0% validity with 15.1% of valid seeds triggering compilation, a 6.4% JIT-trigger rate. Domato produces valid seeds at only 1.5% but every valid Domato program triggers compilation (owing to

the function-heavy structure of its generated code), yielding a 1.5% JIT-trigger rate.

Ordered by JIT-trigger rate, JOTFUZZ (42.1%) leads PolyGlot (6.4%), FlowFusion (1.8%), and Domato (1.5%). This ordering differs notably from the coverage ordering. Aggregate coverage rewards generating many broadly-valid programs, whereas the JIT-trigger rate rewards generating programs that reach the optimizer. JOTFUZZ leads on the second because it is the only fuzzer that enforces JIT preconditions by construction which is the same gap that produces the difference in JIT bug discovery.

Findings reveal that JOTFUZZ substantially outperforms baselines in JIT-trigger rate. The combination of seed validity and JIT-targeted seed generation enables successful JIT activation.

5) System Throughput: Over seven independent 24-hour trials, JOTFUZZ averaged **230.4k seed executions**, exceeding FlowFusion's 47.0k by 4.9× and Domato's 25.7k by 9.0×, while falling short of PolyGlot's 1,855k by 8.1×. All three differences are statistically significant ($p = 0.002$).

Raw throughput, however, is a poor proxy for JIT fuzzing effectiveness, since an execution that never engages the optimizer cannot expose an optimizer bug. Weighting each fuzzer's throughput by its JIT-trigger rate yields the rate at which each actually exercises the JIT: JOTFUZZ produces 97.0k JIT-triggering executions per trial, against PolyGlot's 118.7k, FlowFusion's 0.85k, and Domato's 0.4k. PolyGlot's raw throughput advantage therefore survives this weighting, but its executions reach the optimizer through short, grammar-generated programs that lack the structural richness JOTs impose, and it discovered no JIT bugs over the campaign. JOTFUZZ's throughput is thus sufficient to convert its precondition-satisfying seeds into bug discoveries at a rate no baseline matches.

JOTFUZZ outperforms FlowFusion and Domato in seed throughput. Combined with its high JIT-trigger rate, JOTFUZZ efficiently tests the JIT compiler.

IX. Limitations and Future Work

Type Monomorphism. JOTFUZZ extracts type information when available from the AST and is conservative in its labeling of type stable variables. Because of the polymorphism allowed in dynamically typed languages like JavaScript and PHP, precise monomorphism cannot be statically determined. Consequently, given that JOTs are extracted by static analysis, they are limited in their expression of type stability. While this limits control over explicitly enforcing type stable constraints, this does not prevent JOTFUZZ from generating seeds with type stable variables.

Portability. Our results are specific to the PHP tracing JIT; however, the two-stage formulation is not. JIT-backed runtimes often impose both an executability barrier and an activation barrier, and a fuzzer for such a runtime must clear both. What differs across runtimes is the content of the activation conditions including thresholds and tiering policy. Re-targeting JOTFUZZ would mean re-parameterizing instantiation against those. Our cross-language extraction is evidence that this re-parameterization is tractable in at least one direction; we have not evaluated others.

X. Related Work

Fuzzing. Fuzzing has exposed critical defects across compilers [19], [20], [21], SMT solvers [22], databases [23], and kernels [24], [25], [26]. Interpreters pose a distinct difficulty, since byte-level mutations rarely survive parsing. Atheris [27] applies coverage-guided mutation to CPython extensions. Among JavaScript tools, LangFuzz [28] harvests code fragments from seed programs and splices them into new inputs, blending grammatical structure with mutation. DIE [29] instead constrains its AST mutations to preserve the structural and type aspects that give a seed its bug-triggering value. Tools like Montage [30], [31] learns bug-inducing patterns from a corpus and samples similar programs.

For PHP, Domato [14], Gramatron [32], and NAUTILUS [33] derive PHP from hand-written grammars. However grammars demands substantial manual authorship and must track interpreter updates. PolyGlot [5] generalizes this with annotated BNF, producing semantically validated seeds for several language processors. FlowFusion [2] takes a different route, using lightweight analysis to fuse the data flows of two existing seeds into one.

Optimizer Fuzzing. Where the above optimize for breadth of interpreter coverage, a growing line of work aims at optimization. Fuzzilli and FuzzJIT [34], [35] generate valid JavaScript shaped to drive engines into JIT compilation. Wang et al. [36] treats coverage over optimization paths as its guidance metric and , LOOL [37] repurposes the compiler's own optimization log as a signal that steers generation toward optimization passes, and Optimuzz [38] pairs directed graybox search with translation validation to reach designated passes in LLVM and TurboFan. JITfuzz [39] pursues the same objective for JVM JITs through mutators built to activate optimizations.

Rather than fuzzing the PHP JIT with specialized feedback or targeted mutation, JOTFUZZ targets the JIT before execution rather than during it. Preconditions are satisfied by construction in each generated seed, so no feedback loop is needed to discover them.

XI. Conclusion

We presented JOTFUZZ, a novel fuzzing framework for the PHP JIT compiler that extracts JOTs from JavaScript JIT bugs and generates seeds from these constraints. Our evaluation demonstrates that JOTFUZZ significantly outperforms state-of-the-art fuzzers. JOTFUZZ achieves higher code coverage, and discovered 14 JIT bugs (20 total bugs) while producing JIT-triggering seeds far more efficiently than baselines. Ablation studies further show that both JOTs and dependency slices positively contribute to the overall performance of JOTFUZZ.

Ethics Considerations

Stakeholders. The following stakeholders were considered during the development and testing of JotFuzz: (1) *PHP interpreter developers*: who are responsible for the creation and maintenance of the official PHP interpreter, (2) *PHP application developers*: who use PHP for application development, (3) *end users*: who rely on PHP-powered services, and (4) *security researchers*: who develop tools for automated program testing and verification.

Impacts. This work benefits the PHP ecosystem and its users, enabling PHP developers to find and patch bugs before end users are impacted. It provides a new state-of-the-art tool to better assess both the security and functionality of the official PHP interpreter.

Mitigations. To prevent ethical risks, we took the following steps: (1) all experiments were conducted locally within isolated, sandboxed environments with proper permissions and (2) all discovered vulnerabilities were responsibly disclosed to the PHP development team in accordance with their established disclosure procedure.

Decision. We carefully considered the ethical implications of this work through every step of development and deployment. We conclude that the benefits of this research (enhanced PHP security, new state-of-the-art security techniques) outweigh the mitigated risks. Thus, the conducting and publishing of this research is ethically justified and in the public interest.

References

[1] "Php manual," https://www.php.net/manual/en/index.php, 2025.

[2] Y. Jiang, C. Zhang, B. Ruan, J. Liu, M. Rigger, R. Yap, and Z. Liang, "Fuzzing the php interpreter via dataflow fusion," *arXiv preprint arXiv:2410.21713*, 2024.

[3] Y. Wu, X. Yu, and Z. Qian, "Melting the flesh of PHP's memory hardening," in *35th USENIX Security Symposium (USENIX Security 26)*. Baltimore, MD: USENIX Association, Aug. 2026, pp. 2803–2821. [Online]. Available: https://www.usenix.org/conference/usenixsecurity26/presentation/wu-yifan

[4] CVE Details, "Cve-2024-11235," https://www.cvedetails.com/cve/CVE-2024-11235/, 2024.

[5] Y. Chen, R. Zhong, H. Hu, H. Zhang, Y. Yang, D. Wu, and W. Lee, "One engine to fuzz'em all: Generic language processor testing with semantic validation," in *2021 IEEE Symposium on Security and Privacy (SP)*. IEEE, 2021, pp. 642–658.

[6] C. Click and M. Paleczny, "A simple graph-based intermediate representation," in *Papers from the 1995 ACM SIGPLAN Workshop on Intermediate Representations*, ser. IR '95. New York, NY, USA: Association for Computing Machinery, 1995, p. 35–49. [Online]. Available: https://doi.org/10.1145/202529.202534

[7] The V8 Project Authors, "mjsunit: V8's javascript unit test suite," https://chromium.googlesource.com/v8/v8/+/main/test/mjsunit/, 2026, part of the V8 JavaScript engine.

[8] PHP Group, "Php test suite," https://github.com/php/php-src/tree/master/tests, 2025, accessed: 2025-02-08.

[9] M. Haverbeke and the Acorn contributors, "Acorn: A small, fast, javascript-based javascript parser," https://github.com/acornjs/acorn, 2012–2026, mIT License.

[10] N. Popov, "php-ast: Php abstract syntax tree extension," https://github.com/nikic/php-ast, Aug. 2025, version 1.1.3. [Online]. Available: https://github.com/nikic/php-ast

[11] G. Van Rossum, *The Python Library Reference, release 3.8.2*. Python Software Foundation, 2020.

[12] A. Hazimeh, A. Herrera, and M. Payer, "Magma: A ground-truth fuzzing benchmark," *Proceedings of the ACM on Measurement and Analysis of Computing Systems*, vol. 4, no. 3, p. 1–29, Nov. 2020. [Online]. Available: http://dx.doi.org/10.1145/3428334

[13] J. Metzman, L. Szekeres, L. Simon, R. Sprabery, and A. Arya, "Fuzzbench: an open fuzzer benchmarking platform and service," New York, NY, USA, p. 1393–1403, 2021. [Online]. Available: https://doi.org/10.1145/3468264.3473932

[14] I. Fratric, "The great dom fuzz-off of 2017," 2017. [Online]. Available: https://googleprojectzero.blogspot.com/2017/09/the-great-dom-fuzz-off-of-2017.html

[15] G. Klees, A. Ruef, B. Cooper, S. Wei, and M. Hicks, "Evaluating fuzz testing," 2018. [Online]. Available: https://arxiv.org/abs/1808.09700

[16] "Gcovr: Generate code coverage reports with gcc/gcov," https://gcovr.com/en/stable/, gcovr authors, 2025, accessed: 2025-12-16. [Online]. Available: https://gcovr.com/en/stable/

[17] MITRE Corporation, "Common Weakness Enumeration (CWE)," https://cwe.mitre.org/, 2025, accessed: Month Year.

[18] M. Wen, Y. Wang, Y. Xia, and H. Jin, "Evaluating seed selection for fuzzing JavaScript engines," *Empirical Software Engineering*, vol. 28, no. 6, p. 133, 2023.

[19] K. Even-Mendoza, A. Sharma, A. F. Donaldson, and C. Cadar, "Grayc: Greybox fuzzing of compilers and analysers for c," in *Proceedings of the 32nd ACM SIGSOFT International Symposium on Software Testing and Analysis*, 2023, pp. 1219–1231.

[20] X. Li, X. Liu, L. Chen, R. Prajapati, and D. Wu, "Fuzzboost: Reinforcement compiler fuzzing," in *International Conference on Information and Communications Security*. Springer, 2022, pp. 359–375.

[21] H. Kim, S. Kim, J. Lee, and S. K. Cha, "Asfuzzer: Differential testing of assemblers with error-driven grammar inference," in *Proceedings of the 33rd ACM SIGSOFT International Symposium on Software Testing and Analysis (ISSTA 2024)*, ser. ISSTA '24. Vienna, Austria: Association for Computing Machinery, Sep. 2024, pp. 1099–1111.

[22] M. N. Mansur, M. Christakis, V. Wüstholz, and F. Zhang, "Detecting critical bugs in smt solvers using blackbox mutational fuzzing," in *Proceedings of the 28th ACM joint meeting on European software engineering conference and symposium on the foundations of software engineering*, 2020, pp. 701–712.

[23] Z.-M. Jiang, J.-J. Bai, and Z. Su, "{DynSQL}: Stateful fuzzing for database management systems with complex and valid {SQL} query generation," in *32nd USENIX Security Symposium (USENIX Security 23)*, 2023, pp. 4949–4965.

[24] M. Fleischer, D. Das, P. Bose, W. Bai, K. Lu, M. Payer, C. Kruegel, and G. Vigna, "Actor: action-guided kernel fuzzing," in *Proceedings of the 32nd USENIX Conference on Security Symposium*, ser. SEC '23. USA: USENIX Association, 2023.

[25] H. Sun, Y. Shen, J. Liu, Y. Xu, and Y. Jiang, "KSG: Augmenting kernel fuzzing with system call specification generation," in *2022 USENIX Annual Technical Conference (USENIX ATC 22)*. Carlsbad, CA: USENIX Association, Jul. 2022, pp. 351–366. [Online]. Available: https://www.usenix.org/conference/atc22/presentation/sun

[26] J. Xu, X. Zhang, S. Ji, Y. Tian, B. Zhao, Q. Wang, P. Cheng, and J. Chen, "Mock: Optimizing kernel fuzzing mutation with context-aware dependency," in *Proceedings of the 31st Annual Network and Distributed System Security Symposium (NDSS 2024)*. San Diego, CA, USA: The Internet Society, 2024.

[27] G. O. S. Team, "Atheris: A coverage-guided, native python fuzzer," accessed: 2025-10-18. [Online]. Available: https://github.com/google/atheris

[28] C. Holler, K. Herzig, and A. Zeller, "Fuzzing with code fragments," in *21st USENIX Security Symposium (USENIX Security 12)*. Bellevue, WA: USENIX Association, Aug. 2012, pp. 445–458. [Online]. Available: https://www.usenix.org/conference/usenixsecurity12/technical-sessions/presentation/holler

[29] S. Park, W. Xu, I. Yun, D. Jang, and T. Kim, "Fuzzing javascript engines with aspect-preserving mutation," in *2020 IEEE Symposium on Security and Privacy (SP)*, 2020, pp. 1629–1642.

[30] S. Lee, H. Han, S. K. Cha, and S. Son, "Montage: A neural network language {Model-Guided}{JavaScript} engine fuzzer," in *29th USENIX Security Symposium (USENIX Security 20)*, 2020, pp. 2613–2630.

[31] H. Xu, Y. Wang, Z. Jiang, S. Fan, S. Fu, and P. Xie, "Fuzzing javascript engines with a syntax-aware neural program model," *Computers & Security*, p. 103947, 2024.

[32] P. Srivastava and M. Payer, "Gramatron: effective grammar-aware fuzzing," in *Proceedings of the 30th ACM SIGSOFT International Symposium on Software Testing and Analysis*, ser. ISSTA 2021. New York, NY, USA: Association for Computing Machinery, 2021, p. 244–256. [Online]. Available: https://doi.org/10.1145/3460319.3464814

[33] C. Aschermann, T. Frassetto, T. Holz, P. Jauernig, A.-R. Sadeghi, and D. Teuchert, "Nautilus: Fishing for deep bugs with grammars." in *NDSS*, 2019.

[34] S. Groß, S. Koch, L. Bernhard, T. Holz, and M. Johns, "Fuzzilli: Fuzzing for javascript jit compiler vulnerabilities," in *NDSS*, 2023. [Online]. Available: https://www.ndss-symposium.org/ndss-paper/fuzzilli-fuzzing-for-javascript-jit-compiler-vulnerabilities/

[35] J. Wang, Z. Zhang, S. Liu, X. Du, and J. Chen, "FuzzJIT: Oracle-Enhanced fuzzing for JavaScript engine JIT compiler," in *32nd USENIX Security Symposium (USENIX Security 23)*, 2023, pp. 1865–1882. [Online]. Available: https://www.usenix.org/conference/usenixsecurity23/presentation/wang-junjie

[36] J. Wang, Y. Kang, C. Wu, Y. Hu, Y. Sun, J. Ren, Y. Lai, M. Xie, C. Zhang, T. Li, and Z. Wang, "OptFuzz: Optimization path guided fuzzing for JavaScript JIT compilers," in *33rd USENIX Security Symposium (USENIX Security 24)*, 2024, pp. 865–882. [Online]. Available: https://www.usenix.org/conference/usenixsecurity24/presentation/wang-jiming

[37] F. Schwarcz, F. Berlakovich, G. Barany, and H. Mössenböck, "LOOL: Low-overhead, optimization-log-guided compiler fuzzing (registered report)," in *3rd ACM International Fuzzing Workshop (FUZZING)*, 2024, pp. 42–51.

[38] J. Kwon, B. Jang, J. Lee, and K. Heo, "Optimization-directed compiler fuzzing for continuous translation validation," *Proceedings of the ACM on Programming Languages*, vol. 9, no. PLDI, pp. 627–650, 2025.

[39] M. Wu, M. Lu, H. Cui, J. Chen, Y. Zhang, and L. Zhang, "JITfuzz: Coverage-guided fuzzing for JVM just-in-time compilers," in *45th International Conference on Software Engineering (ICSE)*, 2023, pp. 56–68.